



مدرس: رامتین خسروی

طراحان: فرجاد فلاح، پارسا دقیق، مانی حسینی،

امیرعلی دهقانی، پریا پاسه‌ورز، امیرحسین

عارف‌زاده، آرتین توسلی، صادق صمدی

مهلت تحویل: جمعه ۳۰ آبان ۱۴۰۴، ساعت ۲۳:۵۹

مقدمه

هدف از این تمرین آشنایی شما با مفاهیم اولیه **طراحی شیء‌گرا^۱** و طراحی یک نرم‌افزار به کمک آن است. از آنجایی که استفاده و درک درست این مفاهیم در پیاده‌سازی سایر تمرین‌های این درس لازم است، پیشنهاد می‌شود به این تمرین زمان کافی را اختصاص دهید.

استفاده از نرم‌افزاری که در این تکلیف می‌نویسید از طریق وارد کردن تعدادی دستور ورودی و دریافت خروجی صورت می‌گیرد. **توصیه می‌شود نوشتن برنامه را به صورت دستور به دستور جلو ببرید.** یک دستور را کامل کنید، درستی کد خود را بیازمایید و بعد به پیاده‌سازی دستور بعد بپردازید. به این ترتیب حتی اگر در مهلت تعیین شده موفق به تکمیل تمام تکلیف نشدید، بخشی از کارکردها را کامل کرده‌اید. با این هدف، ترتیب مناسبی از دستورها را برای پیاده‌سازی تعریف کنید. به عبارت دیگر، اولین دستور که در اینجا توضیح داده شده لزوماً مناسب‌ترین دستور برای شروع پیاده‌سازی نیست.

^۱ Object-Oriented Design

یوتی بانک

شرح مسئله

پارسا که برای تامین مالی شرکت جدیدش به دنبال وام است، مدام از بانکی به بانک دیگر می‌رود ولی تاکنون موفق به اخذ وام نشده است. دانشگاه تهران به تازگی بانکی تأسیس کرده اما هنوز اطلاعات را به صورت دستی مدیریت می‌کند و به همین دلیل فرآیند اعطای وام بسیار کند شده است. پارسا از شما می‌خواهد برای این بانک سیستمی طراحی کنید تا سرعت بررسی درخواست‌های وام سریع‌تر شود که او بتواند هر چه سریع‌تر وام خود را بگیرد. در این تمرین، شما ابتدا اطلاعات قبلی مشتریان و حساب‌ها را وارد سیستم می‌کنید و در ادامه با پیاده‌سازی دستورات و گزارش‌های مختلف فرآیندهای بانک را تسریع می‌کنید.

فایل‌های ورودی

اطلاعات مشتریان

یوتی‌بانک از قبل مشتریانی داشته که اطلاعات آن‌ها باید وارد سیستم شود. در یک فایل CSV اطلاعات مشتریان شامل نام، نوع مشتری (حقیقی یا حقوقی) و شماره حساب‌های او (هر مشتری می‌تواند چند حساب مختلف در یوتی‌بانک داشته باشد) آمده‌اند. تضمین می‌شود که در نام مشتری‌ها، کاراکتر فاصله² وجود ندارد. در لیست شماره حساب‌ها، کاراکتر «؛»³ نقش جداکننده بین شماره حساب‌ها را دارد. همچنین تضمین می‌شود که شماره حساب‌ها و نام مشتریان یکتا باشند و هر شماره حساب تنها برای یک مشتری باشد.

نام ویژگی	توضیحات	نوع داده
Name	نام مشتری	رشته
Type	نوع مشتری	رشته ("Corporate","Individual")
AccountNumbers	شماره حساب‌های مشتری	عدد طبیعی

² Space

³ Semicolon

نمونه فایل CSV
Name , Type , AccountNumbers IranKhodro_Inc , Corporate , 1126750 ; 1127850 Parsa , Individual , 1235075 ; 2259754 ; 2349723 Paria , Individual , 3121410 ; 5946234

اطلاعات حساب‌ها

در این فایل CSV اطلاعات حساب‌ها شامل شماره حساب، نرخ سود ماهانه و موجودی فعلی آن حساب آمده‌اند. تضمین می‌شود که شماره حساب‌ها یکتا باشند. سود ماهانه یک عدد طبیعی است. برای مثال ۵ به معنی آن است که بعد از هر بار پرداخت سود ماهانه، موجودی حساب ۵ درصد افزایش پیدا می‌کند. تضمین می‌شود اطلاعات همه شماره حساب‌هایی که در فایل مشتریان آمده است، در این فایل موجود باشد.

نام ویژگی	توضیحات	نوع داده
AccountNumber	شماره حساب	عدد طبیعی
InterestRate	درصد سود ماهانه حساب	عدد طبیعی
Balance	موجودی حساب	عدد طبیعی

نمونه فایل CSV
AccountNumber , InterestRate , Balance 1126750 , 3 , 125000750 1235075 , 4 , 999999999 3121410 , 2 , 13256713

نحوه اجرای برنامه

برای اجرای برنامه، مسیر فایل‌های ورودی به ترتیب (ابتدا مسیر فایل مشتریان و سپس مسیر فایل حساب‌ها) از طریق آرگومان خط فرمان⁴ به برنامه داده می‌شود. برای آشنایی با این نوع آرگومان‌ها، می‌توانید [این لینک](#) را مشاهده کنید.

قالب ورودی
<code>./UTBank </path/to/clients/file> </path/to/accounts/file></code>

دستورات

افتتاح حساب

با وارد شدن این دستور یک حساب جدید با شماره حساب و درصد سود تعیین شده ایجاد و به مشتری مورد نظر اختصاص پیدا می‌کند. موجودی این حساب جدید برابر صفر است. تضمین می‌شود که شماره حساب جدید، معتبر بوده و تکراری نمی‌باشد. همچنین تضمین می‌شود که کاربر مورد نظر در سیستم موجود باشد.

قالب ورودی
<code>open_account <client_name> <new_account_number> <interest_rate></code>
قالب خروجی
<code>An account with account number <new_account_number> was created and assigned to <client_name>.</code>

نمونه ورودی
<code>open_account Farjad 4486257 3</code>
نمونه خروجی
<code>An account with account number 4486257 was created and assigned to Farjad.</code>

⁴ Command line

انتقال وجه

هر کاربر بانک می‌تواند با استفاده از سامانه یوتی‌بانک مبلغ مشخصی را از حساب خود به حسابی دیگر انتقال دهد. اگر مقدار مبلغ وجه انتقالی از موجودی حساب مبدأ بیشتر باشد، باید پیغام خطای مناسبی نمایش داده شود. تضمین می‌شود که شماره حساب مبدأ و مقصد یکسان نباشند و هر دو نیز مقداری معتبر داشته باشند. همچنین مبلغ انتقالی همواره عدد طبیعی خواهد بود.

قالب ورودی
transfer_money <origin_account_number> <dest_account_number> <amount>
قالب خروجی
<amount> transferred successfully from <origin_client_name> to <dest_client_name>. Insufficient balance.

نمونه ورودی ۱
transfer_money 54 3 20000
نمونه خروجی ۱
20000 transferred successfully from Parsa to Farjad.

در صورتی که مقدار تراکنش از موجودی حساب مبدا بیشتر باشد، کاربر با خطا مواجه می‌شود:

نمونه خروجی ۲
Insufficient balance.

گرفتن وام

به کمک این دستور، کاربر می‌تواند از بانک وام بگیرد. اگر بار اولی است که از بانک وام می‌گیرد، بانک به صورت کاملاً سخاوتمندانه به اندازه نصف موجودی حساب به کاربر وام می‌دهد. اما در دفعات بعد مبلغ وام با توجه به امتیاز وفاداری کاربر محاسبه می‌شود. امتیاز وفاداری هر شخص در ابتدا برابر با ۵۰ است. با نحوه تغییر این امتیاز در بخش پرداخت اقساط وام آشنا می‌شوید. مقدار وام تخصیص داده شده به کاربر با رابطه زیر محاسبه می‌شود:

$$L_{new} = \lfloor L_{old} * (1 + \frac{P}{100}) \rfloor$$

$$L_0 = \lfloor 0.5 \times balance \rfloor$$

L_{new} مقدار وام جدید، L_{old} مقدار وام قبلی کاربر و p مقدار امتیاز وفاداری کاربر است. مقدار نهایی وام همواره عددی طبیعی است و در صورتی که عدد حاصل اعشاری بود باید به عدد طبیعی قبلی خود قطع شود. L_0 نیز مقدار اولین وام مشتری و $balance$ موجودی حسابی است که مشتری می‌خواهد با استفاده از آن وام بگیرد. برای این دستور به ترتیب به شناسه وام، شماره حساب مورد نظر (برای محاسبه مقدار وام اولیه برای اولین وام) و تعداد اقساط وام نیاز داریم. بعد از اجرای این دستور موجودی حساب معرفی شده به اندازه مقدار وام تخصیص یافته افزایش پیدا می‌کند.

توجه داشته باشید هر کاربر می‌تواند چندین حساب بانکی داشته باشد و از هر کدام از حساب‌های خود درخواست وام کند. اما مبلغ وام بر اساس آخرین وام دریافتی کاربر محاسبه می‌شود، نه آخرین وام حساب. یعنی بانک همیشه آخرین وامی که خود کاربر با هر حسابی گرفته است را در نظر می‌گیرد و با استفاده از آن، مبلغ وام جدید را تعیین می‌کند. شناسه وام یک عدد طبیعی و یکتاست که در هنگام ایجاد یک وام جدید مشخص می‌شود و از آن به بعد به کمک آن می‌توان وام مورد نظر را در سیستم مشخص کرد.

قالب ورودی
grant_loan <loan_id> <account_number> <num_of_payments>
قالب خروجی
A <loan_ammount> loan granted to <client_name> successfully.

نمونه ورودی
grant_loan 123 54 6
نمونه خروجی
A 518 loan granted to Parsa successfully.

پرداخت قسط

در این قسمت کاربر می‌تواند یکی از قسط‌های وامی که اخذ کرده را پرداخت کند. تضمین می‌شود که بیشتر از تعداد اقساط وام، پرداختی صورت نگیرد. مبلغ قسطی که پرداخت می‌شود برابر حاصل تقسیم مبلغ کل وام به تعداد اقساط آن است. اگر مقدار قسط اعشاری شد به عدد طبیعی کوچک‌تر قطع شود. البته در صورتی که قسط پایانی باشد کل مبلغ باقی‌مانده به عنوان مبلغ قسط در نظر گرفته می‌شود. برای مثال اگر کاربر وامی با مبلغ اولیه ۲۰ و با تعداد ۳ قسط گرفته باشد مبلغ دو قسط اول برابر ۶ و مبلغ قسط سوم برابر ۸ خواهد بود. هر بار که یک قسط پرداخت می‌شود، امتیاز وفاداری کاربر با توجه به فرمول زیر تغییر می‌کند:

$$p_{new} = \lfloor (\alpha \times p_{old} + (1 - \alpha) \times (\frac{-Delay}{60} \times 100 + 50)) \rfloor$$

در اینجا p_{new} امتیاز وفاداری جدید است، p_{old} امتیاز وفاداری قبلی و $Delay$ نیز تعداد روزهای دیرکرد پرداخت وام است. مقدار α هم اگر حساب حقیقی باشد ۰.۱ و اگر حقوقی باشد ۰.۲ است. توجه کنید که اگر کسی قسطی را زودتر از موعد پرداخت کند، دیرکرد، مقداری منفی خواهد داشت. برای مثال اگر قسطی ۵ روز زودتر پرداخت شود، دیرکرد برابر ۵- و اگر ۲ روز دیرتر از موعد قسط پرداخت شود، دیرکرد برابر ۲+ می‌شود. تضمین می‌شود صاحب حساب و صاحب وام یکسان باشند. همانطور که در فرمول تغییر امتیاز وفاداری مشخص است، مقدار آن همواره عددی طبیعی خواهد بود و در غیر این صورت به نزدیک‌ترین عدد طبیعی کوچک‌تر قطع می‌شود.

قالب ورودی
pay_installment <loan_id> <account_number> <delay>
قالب خروجی
<client_name> successfully paid <amount_of_installment> for one installment.

نمونه ورودی
pay_installment 123 54 -5
نمونه خروجی
Parsa successfully paid 159 for one installment.

پرداخت سود

با این دستور بانک تصمیم می‌گیرد تا سود ماهانه را به تمامی حساب‌های مشتریان واریز کند. درصد سود هر حساب مشخص است و به همان نسبت به موجودی حساب افزوده می‌شود. برای مثال اگر درصد سود یک حساب ۵ باشد و موجودی آن نیز برابر ۲۰۰ باشد، موجودی جدید پس از پرداخت سود برابر ۲۱۰ خواهد بود. دقت شود بعد از وارد شدن این دستور، سود تمامی حساب‌ها باید محاسبه و به موجودیشان اضافه شود. در نظر داشته باشید که موجودی حساب همواره باید عددی طبیعی باشد و در غیر این صورت باید به عدد طبیعی کوچکتر قطع شود.

$$Balance_{new} = [Balance_{old} \times (1 + (\frac{interest\ rate}{100}))]$$

در این رابطه، $Balance_{new}$ موجودی بعد از پرداخت سود، $Balance_{old}$ موجودی قبل از پرداخت سود و $interest\ rate$ نرخ سود ماهانه حساب است. حساب‌ها بر اساس شماره حساب و به صورت صعودی در خروجی چاپ می‌شوند.

قالب ورودی
pay_interest
قالب خروجی
Interest paid successfully, new balances: New balance of <account_number_1> = <account_1_balance> New balance of <account_number_2> = <account_2_balance> ...

نمونه ورودی
pay_interest
نمونه خروجی
Interest paid successfully, new balances: New balance of 4486257 = 158000 New balance of 6586156 = 3250 New balance of 7953132 = 999999 New balance of 8125631 = 1235910

دستورات گزارش‌گیری

در همه گزارش‌گیری‌ها، تمام داده‌های مرتبط با کاربر باید در نظر گرفته شوند. به طور مثال، اگر کاربر چندین وام با حساب‌های مختلف داشته باشد، تمامی وام‌ها در گزارش وام‌ها و محاسبه بدهی کل یا باقی‌مانده وام‌ها اثر خواهند داشت.

در محاسبه مجموع بدهی‌ها یا تعداد اقساط پرداخت‌شده، تمام دفعاتی که کاربر وام دریافت کرده یا قسط پرداخت کرده باید لحاظ شوند، حتی اگر برخی وام‌ها هنوز به پایان نرسیده باشند. توجه کنید که قالب خروجی (از جمله املای کلمات و خطوط خالی) باید برای تمام دستورات در پیاده‌سازی شما درست باشد.

گزارش بزرگ‌ترین بدهکاران

با وارد کردن این دستور، گزارشی از تعداد مشخص شده از مشتریانی که بیشترین مجموع مبلغ باقی‌مانده وام را دارند نمایش داده می‌شود. مجموع مبلغ باقی‌مانده وام‌ها برای یک مشتری برابر مجموع مبلغ وام‌هایی است که او در تمامی از حساب‌هایش گرفته است. تضمین می‌شود به تعداد مشخص شده مشتری در سامانه وجود دارد که بدهی داشته باشند. ترتیب نمایش بر اساس بیشترین مبلغ بدهی به صورت نزولی است و اگر میزان بدهی دو کاربر یکسان بود، بر اساس ترتیب حروف الفبا در اسم کاربر به صورت صعودی مرتب شوند.

قالب ورودی
report top_debtors <count>
قالب خروجی
Top <count> debtors report:

```

1.
<client_name>
Total Remaining Debt: <remaining_debt>
Number of Active Loans: <num_of_active_loans>
Largest Loan ID: <loan_id_with_max_debt>
Largest Loan Remaining Amount: <largest_remaining_amount>

2.
<client_name>
Total Remaining Debt: <remaining_debt>
Number of Active Loans: <num_of_active_loans>
Largest Loan ID: <loan_id_with_max_debt>
Largest Loan Remaining Amount: <largest_remaining_amount>

...

```

اگر یک کاربر چند وام با مقدار بدهی باقی‌مانده یکسان داشته باشد و این مقدار، بیشترین بدهی باقی‌مانده در میان تمام وام‌های او باشد، در این صورت وامی که شناسه‌ی بزرگتری دارد به عنوان <loan_id_with_max_debt> در خروجی در نظر گرفته می‌شود.

نمونه ورودی
report top_debtors 3
نمونه خروجی
Top 3 debtors report: 1. Aidin Total Remaining Debt: 4000 Number of Active Loans: 1 Largest Loan ID: 123 Largest Loan Remaining Amount: 4000

2.
Kourosh
Total Remaining Debt: 2500
Number of Active Loans: 2
Largest Loan ID: 88
Largest Loan Remaining Amount: 2000

3.
Parsa
Total Remaining Debt: 1000
Number of Active Loans: 1
Largest Loan ID: 90
Largest Loan Remaining Amount: 1000

گزارش مشتریان خوش حساب

با وارد کردن این دستور، گزارشی از تعداد مشخص شده از مشتریان خوش حساب نمایش داده می‌شوند. تضمین می‌شود به تعداد مشخص شده مشتری در سامانه وجود دارد. در این گزارش، بالا بودن امتیاز وفاداری به عنوان معیار اصلی خوش‌حسابی در نظر گرفته شده است. اگر امتیاز وفاداری دو کاربر یکسان بود، بر اساس ترتیب حروف الفبا در اسم کاربر به صورت صعودی مرتب شوند.

منظور از مجموع کل مبلغ وام گرفته شده در قالب خروجی، مجموع مبلغ تمامی وام‌های کاربر (هم فعال و هم غیرفعال) است.

قالب ورودی

```
report top_trusted_clients <count>
```

قالب خروجی

```
Top <count> trusted clients report:
```

```
1.  
<client_name>  
Loyalty Score: <p>
```

Number of Completed Loans: <num_of_completed_loans>

Total Borrowed Amount: <total_borrowed>

2.

<client_name>

Loyalty Score: <p>

Number of Completed Loans: <num_of_completed_loans>

Total Borrowed Amount: <total_borrowed>

...

نمونه ورودی

report top_trusted_clients 3

نمونه خروجی

Top 3 trusted clients report:

1.

Paria

Loyalty Score: 83

Number of Completed Loans: 3

Total Borrowed Amount: 12000

2.

Parisa

Loyalty Score: 75

Number of Completed Loans: 1

Total Borrowed Amount: 3000

3.

Kosar

Loyalty Score: 50

Number of Completed Loans: 2

Total Borrowed Amount: 5000

نکات و نحوه تحویل

- تحویل این تمرین در سامانه گیت‌هاب انجام می‌شود. برای انجام تمرین لطفاً از طریق [این لینک](#) وارد شوید، پس از آن باید شماره دانشجویی خود را انتخاب کنید (دقت کنید که با کمک این شماره دانشجویی به شما نمره خواهیم داد، لطفاً در انتخاب درست شماره دانشجویی حتماً دقت کنید، در صورتی که به مشکل خوردید با دستیاران در ارتباط باشید). پس از آن به صفحه‌ای منتقل می‌شوید که در آنجا می‌توانید تمرین جدید را قبول کنید، پس از قبول کردن تمرین یک مخزن⁵ در [AP Assignments](#) برای شما ساخته می‌شود و باید کدهای خود را در آنجا قرار دهید.
- پس از انجام تمرین و بارگذاری در گیت‌هاب، کد Hash آخرین کامیت⁶ را به همراه شماره دانشجویی خود در سامانه ای‌لرن آپلود کنید (در خط اول شماره دانشجویی، پس از آن از Enter استفاده کنید و به خط بعد بروید و پس از آن Hash آخرین کامیت). نمونه متن خواسته شده در سامانه ای‌لرن (بخش <last_commit_hash> و <sid> را جایگزین کنید):

```
<sid>  
<last_commit_hash>
```

نمونه:

```
810100000  
bad8fbcdfca3b9feb371a31e0c370150aa870b18
```

- دقت کنید که عدم رعایت ساختار گفته شده در آپلود یا تغییر ساختار فایل‌ها در مخزن (می‌توانید به دلخواه خود فایل اضافه کنید و ... اما اسم و ساختار فایل‌هایی که در ابتدا به شما داده می‌شود نباید تغییر کند) باعث کسر ۱۰ درصد از نمره شما خواهد شد.
- پروژه شما باید به صورت چند فایل⁷ و با استفاده از makefile پیاده‌سازی شده باشد. هدف اصلی پروژه یادگیری شی‌گرایی بوده و پیاده‌سازی به صورت چند فایل صرفاً برای آشنایی شما با این مفهوم می‌باشد.
- دقت کنید که فایل makefile باید در صفحه اول مخزن باشد و در پوشه‌ای قرار نداشته باشد و در آن مشخص کنید که از استاندارد c++20 استفاده می‌کنید.
- نام برنامه قابل اجرای شما باید UTBank (بدون هیچ پسوندی مانند exe یا out) باشد و پس از ساخته شدن در کنار makefile قرار بگیرد (داخل پوشه‌ای فایل خروجی ساخته شده را قرار ندهید).

⁵ Repository

⁶ Commit

⁷ multi-file

- درستی برنامه شما از طریق آزمون‌های خودکار سنجیده می‌شود؛ بنابراین پیشنهاد می‌شود که با استفاده از ابزارهایی مانند diff خروجی برنامه خود را با خروجی‌هایی که در اختیارتان قرار داده شده است مطابقت دهید.
- هدف این تمرین یادگیری شماست. لطفا تمرین را خودتان انجام دهید. در صورت کشف تقلب مطابق قوانین درس با آن برخورد خواهد شد.

نمرات

ارزیابی پاسخ شما به تکلیف بر اساس موارد زیر انجام خواهد شد.

- تمیزی کد
 - رعایت کردن نام‌گذاری صحیح و انسجام
 - عدم استفاده از متغیرهای سراسری (گلوبال)
 - استفاده مناسب از متغیرهای ثابت به جای Magic Value-ها
- درستی کد
 - آزمون‌های خودکار
- طراحی
 - شکستن به کلاس‌های مناسب و تخصیص مسئولیت‌های صحیح به هر کلاس و استفاده از Header Guard-ها
 - جداسازی منطق کد از ورودی/خروجی
 - رعایت سطح دسترسی (public/private) در ویژگی‌های کلاس
 - ساختاردهی کد در قالب توابع/متدهای کوتاه که فقط یک کار را انجام می‌دهند
- گیت و گیت‌هاب
 - استفاده از commit message‌های مناسب
 - هر کامیت یک کار مشخص انجام بدهد و کامیتی چندین کار انجام ندهد.

دقت کنید که موارد ذکر شده لزوماً کل نمره شما را تشکیل نمی‌دهند و ممکن است با تغییراتی همراه باشند.